

Sistema Inteligente Para la Asistencia en el Desarrollo Estructurado de Código Mediante Spec- Driven Development

Memoria del Trabajo de Final de Grado

Autor: Adrià Cebrián Ruiz
Director: Luis Jonás Cuervo Derqui
Ponente: David Garcia Soriano

Facultat d'Informàtica de Barcelona (FIB)
Menció de Computació — Grau en Enginyeria Informàtica

Junio 2026

9. Cambios en el Alcance del Proyecto

9.1. Motivación del cambio

La planificación inicial del proyecto partía de un conocimiento superficial del ecosistema de herramientas de asistencia al desarrollo existentes. En el GEP se identificaron herramientas como Spec Kit, AgentOS o los asistentes comerciales (Copilot, Cursor), pero sin profundizar en el grado de extensibilidad que estos entornos ofrecían. Durante la fase de Trabajo Previo (TP1), al investigar en detalle la arquitectura de los asistentes de código CLI más avanzados, se descubrió que el ecosistema era considerablemente más rico y maduro de lo que se había anticipado.

En concreto, herramientas como ClaudeCode y OpenCode ofrecen mecanismos nativos de extensibilidad —plugins, hooks de ciclo de vida y el protocolo MCP (Model Context Protocol)— que permiten definir flujos de trabajo de programación estructurados de forma composable, sin necesidad de construir un asistente desde cero. Lo que en la planificación inicial se concebía como un problema de ingeniería complejo (imponer un flujo Spec-First sobre un agente de código) resultó ser un problema de configuración e integración, abordable dentro del alcance de un TFG.

Este descubrimiento, unido a la evolución de las necesidades de Colb.Ai durante el período de desarrollo, fue el detonante del cambio de alcance. La empresa reconoció que el problema real al que se enfrentaba no era la ausencia de herramientas de generación de código —de eso ya había suficiente oferta—, sino la falta de un marco de trabajo que garantizase una estructura de proyecto consistente y trazable con independencia del desarrollador o del asistente concreto que se utilizase. Dicho de otro modo: a Colb.Ai no le importa cómo cada desarrollador aplica las tareas del día a día, sino que todos los proyectos arranquen con una definición clara de su arquitectura, mantengan actualizado un conjunto de documentos de referencia (la constitución del proyecto y el archivo features.md), y que cualquier nueva funcionalidad sea coherente con esas bases antes de ser implementada.

Ante este rediseño del problema, en consenso con la empresa y el director del proyecto, se tomó la decisión de redirigir el foco del TFG hacia una contribución más ajustada a las necesidades reales de Colb.Ai: diseñar, integrar y evaluar el flujo de trabajo completo que cubre el ciclo de vida de un proyecto software, desde la toma de requerimientos hasta el seguimiento del cliente, aprovechando las capacidades de extensibilidad de los asistentes existentes en lugar de construir uno nuevo.

9.2. Nuevos objetivos del proyecto

El proyecto pasa a estructurarse en torno a tres pilares complementarios, cada uno de los cuales aborda una fase diferente del ciclo de vida de un proyecto de software:

Toma de Requerimientos: Diseñar e implementar un flujo de trabajo asistido por IA que, a partir de la transcripción de una reunión con el cliente, sea capaz de generar un plan de implementación estructurado, explorando automáticamente los frameworks y tecnologías relevantes mediante búsqueda semántica en la web.

Spec Driven Development (Estructura del repositorio): Integrar y configurar los asistentes de código de tipo CLI (ClaudeCode y OpenCode) con un conjunto de herramientas, plugins y workflows personalizados que impongan un flujo de trabajo Spec-First al equipo de desarrollo, garantizando que cada implementación esté precedida por una especificación validada y revisada.

Seguimiento del cliente: Automatizar la generación de documentación de seguimiento post-reunión, produciendo de forma estructurada el resumen de lo acordado y los próximos pasos para ambas partes, a partir de la transcripción de las reuniones periódicas con el cliente.

9.3. Continuidad con la planificación inicial

Este cambio de alcance no supone un abandono de los objetivos originales del GEP, sino una evolución coherente de los mismos. Los sub-objetivos de coherencia arquitectónica, trazabilidad entre especificación y código, y modularidad del sistema se mantienen intactos y se materializan ahora a través de la configuración de los asistentes existentes en lugar de mediante el desarrollo de uno propio.

Los riesgos identificados en el GEP (vendor lock-in, gestión de la ventana de contexto, fricción en el bucle de desarrollo) se mantienen vigentes y son abordados específicamente en el diseño de los módulos descritos en los capítulos siguientes. En concreto, el riesgo de vendor lock-in se mitiga mediante la integración paralela de ClaudeCode y OpenCode, tal como se describe en el capítulo 11.

10. Toma de Requerimientos

La toma de requerimientos es la primera fase del ciclo de vida de un proyecto de software. En este contexto, el objetivo es transformar las necesidades expresadas por el cliente en conversaciones informales en documentos de especificación técnica utilizables por el equipo de desarrollo. El proceso se articula en dos módulos independientes y secuenciales: en primer lugar, el módulo de transcripción convierte las reuniones en texto estructurado; a continuación, el módulo de procesamiento analiza ese texto y genera el plan de implementación. Cada módulo dispone de dos implementaciones complementarias, con el propósito de no depender de un único proveedor.

10.1. Módulo de transcripción

Para que el módulo de procesamiento pueda operar, es necesario disponer previamente de una transcripción de la reunión con el cliente. Dependiendo del tipo de reunión, la forma de obtenerla difiere.

10.1.1. Reuniones telemáticas

Las principales plataformas de videoconferencia (Microsoft Teams, Google Meet, Zoom, entre otras) ofrecen transcripción automática integrada. Dado que cada participante habla desde su propio dispositivo y micrófono, la atribución de fragmentos de texto a cada hablante (diarización) es gestionada de forma nativa por la plataforma, sin necesidad de ningún procesamiento adicional. Para las reuniones telemáticas, se utiliza directamente la transcripción exportada por la plataforma correspondiente como entrada al módulo de procesamiento.

10.1.2. Reuniones presenciales

En el caso de las reuniones presenciales, no existe ninguna plataforma que genere una transcripción de forma automática. Para cubrir este caso, se graba el audio de la reunión y se procesa a posteriori mediante Azure Speech Service, el servicio de transcripción de audio proporcionado por el proveedor de servicios cloud de la empresa.

La característica técnica diferencial en este contexto es la diarización (speaker diarization): el servicio no transcribe el audio como un bloque de texto continuo, sino que identifica a cada participante de forma independiente y etiqueta cada fragmento de texto con el hablante que lo pronunció. Esta atribución es especialmente relevante para la toma de requerimientos, ya que permite al módulo de procesamiento distinguir qué peticiones o restricciones provienen del cliente y cuáles son aclaraciones o comentarios del equipo técnico, mejorando la calidad del plan de implementación resultante.

10.2. Módulo de procesamiento de requerimientos

Una vez disponible la transcripción (producida por cualquiera de las vías descritas en la sección anterior), el módulo de procesamiento la analiza y genera un plan de implementación estructurado. Este módulo también dispone de dos implementaciones: un enfoque basado en la plataforma Claude Cwork y una implementación propia en Python.

10.2.1. Enfoque mediante Claude Cowork

Claude Cowork es una herramienta de Anthropic orientada a la automatización de tareas en entorno de escritorio. Su modelo de uso se basa en el concepto de Skill: un workflow de automatización configurable que combina un prompt especializado con conexiones a herramientas y servicios externos.

Una skill define la entrada que recibe (por ejemplo, una transcripción de texto), las herramientas que puede invocar durante su ejecución (búsqueda en internet, lectura y escritura de archivos, llamadas a APIs externas, ejecución de scripts) y el formato de la salida que produce. A diferencia de un prompt ordinario enviado en una conversación, una skill encapsula el flujo de trabajo completo y puede reutilizarse de forma consistente en cualquier proyecto sin necesidad de redefinirlo.

Se ha configurado una skill específica para la toma de requerimientos que implementa el siguiente flujo:

1. El usuario proporciona a la skill la transcripción de la reunión con el cliente, notas informales o un resumen de las necesidades expresadas.
2. La skill procesa el contexto y genera una visión global del proyecto (big picture), identificando los componentes principales del sistema, las posibles tecnologías a emplear y las áreas de incertidumbre que requieren aclaración.
3. Mediante las herramientas de búsqueda en internet integradas en Claude Cowork, el agente investiga las tecnologías propuestas y contrasta alternativas.
4. El agente presenta al usuario las opciones tecnológicas encontradas para que seleccione las preferidas. El resultado es un documento de especificación inicial estructurado, listo para incorporarse al repositorio como punto de partida del desarrollo.

Este enfoque es el preferido cuando se dispone de acceso a la plataforma Claude Cowork, dado que la integración con las herramientas de búsqueda y la calidad del razonamiento son superiores a los de la implementación propia.

10.2.2. Implementación propia

Con el objetivo de disponer de una alternativa de coste reducido e independiente de suscripciones comerciales, se ha desarrollado una aplicación Python que replica el proceso anterior. La aplicación está estructurada en módulos desacoplados y orquestada mediante LangChain.

Arquitectura

La aplicación se compone de cuatro módulos: el transcriptor de audio (speech_transcriptor), el preprocesador de transcripciones (transcription_preprocessing), el gestor de issues de GitHub (issue_manager) y la interfaz de usuario (frontend.py). El módulo central es el preprocesador, que implementa la clase TranscriptionProcessor y coordina toda la lógica de negocio.

El diseño emplea dos modelos de lenguaje con roles diferenciados: un modelo económico (Azure OpenAI) para las operaciones de alta frecuencia que no

requieren razonamiento profundo, y un modelo de mayor capacidad (Google Gemini) para las tareas de síntesis y razonamiento arquitectónico. Esta estrategia de dos niveles optimiza el coste sin sacrificar la calidad del resultado final.

Pipeline de procesamiento

El flujo de procesamiento sigue cinco fases secuenciales:

- 1- Chunking y filtrado: la transcripción se divide en fragmentos de aproximadamente 25.000 caracteres con solapamiento. El modelo económico procesa cada fragmento y extrae únicamente el contenido relevante (requerimientos funcionales, decisiones técnicas), descartando saludos, divagaciones y contenido irrelevante.
- 2- Generación del plan de investigación: el modelo de mayor capacidad analiza el texto filtrado e identifica las decisiones técnicas pendientes (elección de base de datos, framework, infraestructura). A partir de este análisis genera un plan de búsquedas estructurado con un máximo de tres queries, para no saturar el límite de créditos gratuitos del servicio de búsqueda.
- 3- Investigación web mediante Tavily: Tavily es una API de búsqueda semántica en internet diseñada específicamente para agentes de IA. A diferencia de los motores de búsqueda convencionales, devuelve no solo enlaces sino también una respuesta sintetizada y fragmentos de contenido optimizados para el consumo por modelos de lenguaje. Las queries generadas en el paso anterior se ejecutan contra la API de Tavily con búsqueda profunda (`search_depth=advanced`) e inclusión de respuesta resumida, limitando los resultados a fuentes del último año. Los resultados se formatean como contexto de investigación estructurado.
- 4- Generación del plan arquitectónico: el modelo de mayor capacidad combina la transcripción original con los resultados de la investigación y produce el documento de arquitectura en formato Markdown. El documento incluye el resumen de objetivos, la propuesta de stack tecnológico justificada con la investigación, los componentes clave del sistema y la hoja de ruta de implementación.
- 5- Extracción de issues: a partir del plan aprobado, el modelo genera una lista estructurada de issues de GitHub (título, descripción, prioridad, tipo) que pueden crearse automáticamente en el repositorio mediante el módulo `issue_manager`. **[Este apartado se ha descartado, he de eliminarlo de la pipeline]**

Bucle de refinamiento con Human in the Loop

Una característica diferencial de la aplicación es la capacidad de refinamiento iterativo en cada fase. El usuario puede modificar el plan de investigación, los issues generados o el plan arquitectónico mediante instrucciones en lenguaje natural. El sistema mantiene un historial de conversación mediante la abstracción `MessagesPlaceholder` de `LangChain`, lo que permite que el modelo tome en cuenta el contexto de intercambios anteriores al procesar cada solicitud de refinamiento. Este mecanismo implementa el principio *Human in the Loop* definido en los objetivos del proyecto.

11. Spec Driven Development: Implementación

Una vez definidos los requerimientos iniciales del proyecto, el siguiente paso es garantizar que el desarrollo de código siga un flujo estructurado y trazable. Este capítulo describe la arquitectura de herramientas y asistentes que se ha configurado para implementar la metodología SDD en el día a día del equipo de desarrollo.

11.1. Asistentes de Programación

Una de las decisiones de diseño fundamentales del sistema es el rechazo a la dependencia exclusiva de un único proveedor de asistencia al código, fenómeno conocido como vendor lock-in. Para mitigar este riesgo, identificado ya en la planificación inicial (Riesgo 2), se ha optado por la integración de dos asistentes de programación de tipo CLI (Command-Line Interface) que operan directamente desde la terminal:

11.1.1. ClaudeCode

ClaudeCode es la herramienta de asistencia a la programación desarrollada por Anthropic. Se trata de un agente CLI que opera directamente en la terminal del desarrollador, con acceso nativo al sistema de archivos del proyecto. ClaudeCode ha establecido el estándar actual en el ámbito de los agentes de programación, introduciendo o popularizando conceptos clave como los agentes especializados, los hooks de ciclo de vida, las skills, los plugins y el protocolo MCP (Model Context Protocol). Al tratarse de un servicio de pago, su uso está sujeto a las condiciones y tarifas de Anthropic.

11.1.2. OpenCode

OpenCode es un asistente de programación de código abierto compatible con modelos de lenguaje locales y remotos. Su arquitectura es funcionalmente equivalente a la de ClaudeCode: opera desde la terminal, tiene acceso al sistema de archivos y es compatible con el protocolo MCP, lo que lo hace interoperable con la mayoría de las herramientas y plugins desarrollados para el ecosistema. Su naturaleza open source y su compatibilidad con modelos locales lo convierten en la alternativa directa a ClaudeCode en escenarios donde se requiere control total sobre los costes o la privacidad de los datos.

11.1.3. Justificación de la doble integración

La utilización de ambas herramientas no es redundante, sino estratégica. Al configurar el sistema de modo que ambos asistentes sean capaces de operar con la misma configuración de proyecto (archivos de instrucciones, plugins y MCPs), se elimina la dependencia de cualquiera de los dos proveedores. Si las condiciones de uno de ellos cambian de forma desfavorable, el equipo puede migrar su flujo de trabajo sin necesidad de reconfigurarlo desde cero.

Adicionalmente, los asistentes de tipo CLI resuelven de forma más limpia los problemas de integración con el entorno de desarrollo que afectan a los asistentes integrados en el IDE, ya que disponen de herramientas de edición, lectura y búsqueda implementadas directamente sobre el sistema de archivos del sistema operativo.

11.1.4. Arquitectura interna: el patrón ReAct y su implementación en TypeScript

Para comprender plenamente cómo se pueden extender y configurar estas herramientas, es conveniente conocer la arquitectura interna sobre la que están construidas. Tanto ClaudeCode como OpenCode son aplicaciones de TypeScript que implementan un bucle de agente basado en el patrón ReAct (Reasoning + Acting), propuesto por Yao et al. (2022) [22] como paradigma estándar para la construcción de agentes basados en modelos de lenguaje.

El patrón ReAct estructura el comportamiento del agente en un ciclo iterativo de tres fases:

Razonamiento (Thought): El modelo recibe el estado actual del contexto y genera un razonamiento interno en el que planifica el siguiente paso. Este razonamiento no es visible para el usuario, pero es el mecanismo por el que el agente decide qué acción tomar.

Acción (Action): El modelo emite una llamada a herramienta estructurada (tool call) en formato JSON, especificando qué herramienta invocar y con qué parámetros. Las herramientas disponibles incluyen la lectura de archivos, la escritura de código y la ejecución de comandos en terminal.

Observación (Observation): El runtime de TypeScript ejecuta la acción sobre el sistema de archivos o la terminal y devuelve el resultado al modelo como nuevo mensaje en el historial. Este resultado se convierte en la entrada del siguiente ciclo.

El runtime que orquesta este bucle está implementado en TypeScript sobre Node.js. Su flujo de ejecución puede describirse de forma simplificada en tres pasos:

- 1- Al iniciarse una sesión, el runtime construye el system prompt combinando las instrucciones globales del asistente con el contenido del archivo de configuración del proyecto (CLAUDE.md o equivalente), y define el conjunto de herramientas disponibles mediante un esquema JSON.
- 2- En cada iteración del bucle, el runtime envía el historial de conversación actualizado a la API del modelo. Si la respuesta contiene llamadas a herramienta, las ejecuta secuencialmente, añade los resultados al historial y lanza una nueva iteración.
- 3- Si la respuesta no contiene llamadas a herramienta, el runtime la presenta al usuario como respuesta final y espera una nueva instrucción, reiniciando el ciclo con el historial acumulado.

Este diseño en TypeScript es la base sobre la que se apoyan todos los mecanismos de extensibilidad descritos en las secciones siguientes: al tratarse de un proceso Node.js estándar, es posible interceptar el bucle en distintos puntos mediante hooks, inyectar herramientas adicionales mediante plugins, y conectar servicios externos mediante el protocolo MCP.

11.2. Mecanismos de extensibilidad

El valor diferencial de los asistentes CLI frente a otras soluciones reside en su arquitectura de extensibilidad. Antes de describir la configuración concreta del sistema, es necesario definir con precisión los conceptos que la componen, ya que estos términos —plugin, MCP, hook, skill, agente— se utilizan con frecuencia de forma imprecisa en la literatura del sector.

11.2.1. Conceptos clave

Hook: Un hook es un callback que se ejecuta automáticamente en momentos específicos del ciclo de vida del agente: al iniciar una sesión (SessionStart), antes o después de editar un archivo, al ejecutar un comando de terminal, etc. Los hooks permiten interceptar el flujo de trabajo del agente e inyectar comportamiento sin que el usuario lo solicite explícitamente. En colbPowers, el hook SessionStart es el mecanismo que inyecta el skill de bootstrap en cada sesión.

Skill: Una skill es un documento Markdown que contiene instrucciones especializadas para el agente sobre cómo comportarse en una tarea concreta. A diferencia de un prompt de usuario, una skill forma parte del contexto del sistema y está disponible para el agente como conocimiento procedimental persistente. Cuando el agente necesita ejecutar una tarea para la que existe una skill (por ejemplo, hacer brainstorming de una feature o escribir un plan), lee ese documento y sigue sus instrucciones.

Agente (subagente): En el contexto de estos asistentes, un agente o subagente es una instancia del modelo de lenguaje creada dinámicamente por el orquestador para ejecutar una tarea concreta en aislamiento. A diferencia del agente principal, el subagente arranca con un contexto específico y sin acceso al historial de conversación del orquestador. Esta separación garantiza que cada tarea se ejecuta con el contexto mínimo necesario, evitando interferencias entre tareas y reduciendo el consumo de tokens.

Plugin: Un plugin es un paquete instalable que agrupa y distribuye extensiones para el asistente: skills, hooks, comandos y agentes. Su función es análoga a la de un paquete npm: encapsula un conjunto coherente de funcionalidades y las hace instalables en cualquier proyecto con un único comando. Los plugins se instalan a nivel de usuario o de proyecto y son la unidad de distribución del ecosistema.

MCP (Model Context Protocol): El MCP es un protocolo estándar abierto que define cómo un proceso externo puede exponer herramientas al agente. Un servidor MCP es un proceso independiente (escrito en cualquier lenguaje) que anuncia un conjunto de herramientas invocables mediante el protocolo. Cuando el agente necesita una capacidad externa —consultar un grafo de código, buscar en la web, leer una base de datos—, el runtime llama al servidor MCP correspondiente, que ejecuta la operación y devuelve el resultado. La separación entre el runtime del agente y el servidor MCP garantiza que cualquier herramienta externa puede integrarse sin modificar el asistente.

11.2.2. Archivo de instrucciones del proyecto

Ambos asistentes se configuran a nivel de proyecto mediante archivos de instrucciones persistentes que actúan como contexto estático del agente. ClaudeCode utiliza un archivo denominado CLAUDE.md ubicado en la raíz del

repositorio; OpenCode utiliza un archivo equivalente en su directorio de configuración. En cada sesión de trabajo, el asistente incorpora automáticamente este archivo en su system prompt, lo que garantiza acceso inmediato a las reglas del proyecto, el stack tecnológico y los estándares de código sin que el desarrollador los proporcione manualmente. El contenido típico incluye la descripción del proyecto, la pila tecnológica, los estándares de código del equipo, las herramientas disponibles y las restricciones que el agente debe respetar.

11.2.3. Plugins y MCPs

Ambos asistentes son extensibles mediante el protocolo MCP. La instalación y configuración de plugins se realiza a nivel de proyecto o de usuario, garantizando que el conjunto de herramientas disponibles sea consistente en todas las sesiones. Cada plugin puede registrar servidores MCP adicionales que amplían el esquema de herramientas del agente; el agente ve estas herramientas adicionales exactamente igual que las nativas (Read, Write, Bash), y puede invocarlas mediante el mismo mecanismo de tool call descrito en la sección 11.1.4.

11.2.4. Hooks de ciclo de vida

ClaudeCode expone un sistema de hooks configurables en un archivo hooks.json que permite registrar callbacks para distintos eventos del ciclo de vida del agente. Los hooks más relevantes para el flujo SDD son SessionStart (inyección de contexto al inicio de sesión) y PreToolUse (intercepción antes de ejecutar una herramienta concreta, útil para forzar consultas al grafo de código antes de cualquier modificación). En OpenCode, estos hooks se implementan mediante callbacks definidos en el plugin JavaScript correspondiente, con semántica equivalente.

11.2.5. Skills y agentes en colbPowers

colbPowers hace un uso central de ambos mecanismos. Las skills definen el comportamiento del orquestador en cada fase del ciclo SDD: el skill using-colbpowers (inyectado en SessionStart) contiene las reglas de orquestación y detección de estado del proyecto; brainstorming y writing-plans instruyen al agente sobre cómo producir los documentos de diseño y planificación; requesting-code-review define el protocolo de auditoría del código generado. Los subagentes se crean dinámicamente cuando una tarea requiere ejecución aislada: el orquestador genera un prompt específico para esa tarea (incluyendo el design.md y el tasks.md relevantes, y excluyendo el resto del historial) y lanza un subagente que opera exclusivamente sobre ese contexto acotado.

11.3. Herramientas de Apoyo al Flujo SDD

La efectividad del flujo SDD depende en gran medida de la calidad del contexto que el agente tiene disponible en cada momento. A continuación se describen las herramientas principales integradas en el sistema.

11.3.1. code-review-graph

code-review-graph es una herramienta de código abierto que utiliza el parser de sintaxis Tree-Sitter para analizar el repositorio y construir un grafo de dependencias entre las distintas entidades de código (funciones, clases, módulos). Este grafo se almacena en una base de datos SQLite local y se expone

al agente a través de un servidor MCP, proporcionando un mecanismo eficiente para navegar la estructura del código sin necesidad de leer todos los archivos del proyecto.

La integración de esta herramienta aborda directamente el problema de gestión de la ventana de contexto identificado en el apartado de riesgos. En pruebas preliminares realizadas sobre un monorepo de Colb.Ai, el agente con code-review-graph integrado necesitó aproximadamente 40.000 tokens para describir la arquitectura del proyecto, frente a los 64.000 tokens requeridos sin la herramienta, lo que supone una reducción del 37,5% en esa prueba concreta. Las capacidades que expone al agente incluyen: análisis de diffs de git para identificar cambios de mayor riesgo, cálculo del radio de impacto (blast radius) de cualquier modificación, identificación de los flujos de ejecución afectados, búsqueda semántica de entidades de código, y obtención de una visión de alto nivel de la arquitectura del sistema.

11.3.2. Herramientas de métricas de complejidad

La disponibilidad del árbol sintáctico abstracto (AST) de Tree-Sitter en la base de code-review-graph permite calcular métricas de calidad de código de forma granular a nivel de función. Las métricas de interés son la Complejidad Ciclomática (número de caminos independientes en el flujo de control de una función), el Volumen de Halstead (medida del vocabulario del código basada en operadores y operandos únicos) y el Índice de Mantenibilidad (combinación logarítmica de las dos métricas anteriores junto con las líneas de código). Estas métricas se calculan durante la fase de construcción del grafo y se almacenan como atributos de cada nodo, quedando disponibles para que el agente las consulte al auditar el código generado.

11.3.3. colbPowers: flujo de trabajo

colbPowers es un plugin instalable en cualquier proyecto que implementa el flujo de trabajo Spec-First de forma autónoma. Una vez instalado, no requiere que el desarrollador recuerde invocar comandos ni seguir un protocolo manual: el propio agente detecta el estado del proyecto al inicio de cada sesión y guía al usuario por el flujo correcto de forma proactiva.

Estructura del proyecto esperada

Para funcionar, colbPowers asume que el proyecto dispone de una plantilla de repositorio con una estructura específica bajo el directorio .specs/memory/. Esta plantilla incluye los documentos base del proyecto y puede contener adicionalmente indicaciones de estilo, planificación, o apuntes y referencias de proyectos anteriores que sirven de contexto al agente:

constitution.md: Documento de alcance global. Define la visión del proyecto, el stack tecnológico, los estándares de código no negociables y las reglas de gobernanza. Es la fuente de verdad sobre cómo se trabaja en este proyecto.

features.md: Roadmap del producto. Lista todas las features aprobadas con su descripción, requisitos funcionales, restricciones técnicas y dependencias. Cualquier nueva funcionalidad debe estar aquí definida antes de poder planificarse.

docs/specs/: Documentos de diseño por feature: descripción del comportamiento esperado, decisiones de arquitectura y justificación técnica de cada implementación.

docs/plans/: Planes de implementación granulares por feature: listas de tareas ordenadas con criterios de aceptación.

Flujo de trabajo autónomo

El flujo de trabajo completo se desarrolla de la siguiente forma cuando el usuario solicita cualquier cambio o nueva funcionalidad:

- 1- Verificación del estado del proyecto: al iniciar la sesión, el agente lee `.specs/memory/` y comprueba si existen la constitución y el roadmap de features. Si alguno de los dos está ausente, el agente inicia de forma proactiva una conversación guiada con el usuario para definirlos, formulando preguntas sobre la visión del proyecto, el stack tecnológico, los estándares de código y las funcionalidades previstas. Este paso se ejecuta una única vez por proyecto.
- 2- Definición interactiva de la especificación: una vez que los documentos base existen, el agente lleva al usuario por un proceso de preguntas estructuradas para definir con precisión la nueva implementación. Se determina qué debe hacer, a cuál de las features del roadmap pertenece, cuáles son los comportamientos esperados, qué restricciones técnicas aplican y qué criterios de aceptación debe cumplir. El objetivo es eliminar toda ambigüedad antes de escribir una sola línea de código.
- 3- Generación de la especificación: cuando el usuario confirma que la definición es correcta, el agente redacta el documento de especificación (`design.md`) y lo guarda en `docs/specs/`. Este documento es la fuente de verdad de la implementación y será consultado en cada paso posterior.
- 4- Generación del plan de implementación: a partir de la especificación, el agente elabora un plan de tareas (`plan.md`) en `docs/plans/` que desglosa la implementación en pasos concretos y ordenados, cada uno con su criterio de aceptación explícito.
- 5- Ejecución iterativa con implementador y revisor: el agente utiliza la herramienta de lista de tareas (TodoWrite) integrada en los asistentes CLI para ejecutar el plan de forma iterativa. Para cada tarea, actúa un subagente implementador que escribe el código, y a continuación un subagente revisor que verifica que el código se ajusta a la especificación, sigue el plan y pasa los tests definidos. Cuando estén disponibles, el revisor utilizará además las métricas de calidad de código para evaluar la calidad de la implementación.

11.3.4. La plantilla de repositorio

La plantilla de repositorio que colbPowers espera encontrar está diseñada para mantenerse ligera y no imponer burocracia innecesaria. Los únicos documentos imprescindibles son `constitution.md` y `features.md`, que el propio agente puede generar desde cero en la primera sesión si no existen. El resto de la estructura (`docs/specs/`, `docs/plans/`) se va poblando de forma orgánica a medida que el

equipo desarrolla features, convirtiéndose en un registro vivo y trazable de todas las decisiones de diseño del proyecto.

Esta aproximación resuelve uno de los problemas identificados en el análisis del estado del arte: a diferencia de herramientas como Spec Kit, que exigen documentación exhaustiva incluso para cambios pequeños, colbPowers adapta el nivel de formalismo a la magnitud de la tarea. Una corrección menor puede implementarse directamente con el agente; una nueva feature importante pasará por el ciclo completo de especificación, planificación y revisión. El agente decide el nivel de rigor apropiado en cada caso, basándose en el contexto del proyecto y la magnitud del cambio solicitado.

11.4. Evaluación y Resultados

Con el objetivo de evaluar empíricamente el impacto de las herramientas desarrolladas, se ha diseñado un experimento controlado que compara tres condiciones de implementación sobre dos casos de uso distintos. Los experimentos están planificados y pendientes de ejecución; esta sección describe el diseño experimental y la estructura de resultados que se completará con los datos obtenidos.

11.4.1. Diseño experimental

El experimento sigue un diseño de tres condiciones (A/B/C) con tres repeticiones por condición ($n=3$), ejecutadas con temperatura 0 para minimizar la varianza estocástica del modelo. Las tres condiciones son:

Condición A — Sin plugins: El agente dispone únicamente de las herramientas nativas del asistente CLI (Read, Write, Edit, Bash, Grep). No se activa ningún plugin ni servidor MCP adicional.

Condición B — Con code-review-graph: El agente dispone de las herramientas nativas más el servidor MCP de code-review-graph. El agente puede consultar el grafo de dependencias, calcular el blast radius de los cambios y realizar búsquedas semánticas sobre el código.

Condición C — Con colbPowers + code-review-graph: El agente opera bajo el flujo SDD completo de colbPowers, con acceso además al servidor MCP de code-review-graph. La tarea se ejecuta siguiendo el ciclo constitution → features → spec → plan → execute → review.

Las métricas recogidas en cada ejecución son: tokens de entrada consumidos, tokens de salida consumidos, número total de tool calls, número de tool calls específicas de code-review-graph, cobertura de la implementación (porcentaje de ítems del enunciado correctamente implementados), y número de regresiones introducidas (cambios en funcionalidad no solicitada).

11.4.2. Caso 1: ConfortMental — cambios cross-cutting en base de código real

El primer caso de uso aplica el experimento sobre un repositorio de producción de Colb.Ai: la aplicación móvil ConfortMental, desarrollada en React Native. La tarea consiste en implementar los cinco cambios globales del documento de revisión de la aplicación (Bloque 1): unificación tipográfica con DM Sans y DM Serif Display,

corrección de contraste de textos secundarios a WCAG AA, sustitución del patrón de validación de formularios por mensajes inline en rojo, humanización y traducción de mensajes de error del sistema, y unificación de las etiquetas de frecuencia de check-in.

Este caso de uso es especialmente adecuado para evaluar code-review-graph porque los cinco cambios son cross-cutting: afectan a múltiples pantallas y componentes de forma transversal. La hipótesis es que el agente bajo la condición B identificará más instancias de cada patrón (mayor cobertura) y consumirá menos tokens de exploración que el agente bajo la condición A, que debe descubrir la estructura del proyecto mediante herramientas estándar.

Métrica	Condición A (sin plugins)	Condición B (code-review-graph)	Condición C (colbPowers + CRG)
Tokens entrada (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Tokens salida (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Tool calls totales (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Tool calls CRG (media)	N/A	[PENDIENTE]	[PENDIENTE]
Cobertura (media %)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]

11.4.3. Caso 2: Aplicación de simulación 3D

El segundo caso de uso aplica el experimento sobre una tarea de nueva implementación desde cero: el desarrollo de una aplicación de simulación 3D en la que, al pulsar una tecla, aparece una esfera en el entorno tridimensional; al hacer clic sobre ella, la esfera explota en una animación de partículas. A diferencia del caso ConfortMental (modificación de una base de código existente), este caso evalúa el comportamiento de las herramientas en la creación de funcionalidad nueva.

Este caso es especialmente relevante para evaluar colbPowers porque la condición C puede ejecutar el ciclo completo de la metodología SDD: el Architect define la constitution del proyecto y las features, el Developer genera el documento de diseño y el plan de tareas, y la ejecución se realiza tarea a tarea con revisión al final. La hipótesis es que la condición C produce código de mayor calidad estructural (menor complejidad ciclomática, mejor índice de mantenibilidad) y mayor trazabilidad entre las decisiones de diseño y el código implementado.

Métrica	Condición A (sin plugins)	Condición B (code-review-graph)	Condición C (colbPowers + CRG)
Tokens entrada (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Tokens salida (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Tool calls totales (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]

Cobertura funcional (media %)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Complejidad ciclomática (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]
Índice de mantenibilidad (media)	[PENDIENTE]	[PENDIENTE]	[PENDIENTE]

12. Seguimiento del Cliente

La tercera dimensión del proyecto aborda la automatización del seguimiento posterior a las reuniones periódicas con el cliente. El objetivo es minimizar el tiempo dedicado a la redacción de documentación de seguimiento y garantizar que los acuerdos alcanzados en cada reunión queden recogidos de forma estructurada y accesible para ambas partes.

12.1. Generación de documentación con Claude Cowork

El flujo implementado mediante Claude Cowork se integra directamente con el proceso de reunión. Al finalizar cada reunión de seguimiento con el cliente, la transcripción o el resumen de lo acordado se proporciona como entrada a una skill de Claude Cowork desarrollada específicamente para este propósito. La skill procesa la información y genera de forma automática un documento Word estructurado que incluye los siguientes apartados: el resumen de los temas tratados en la reunión, las decisiones tomadas y los acuerdos alcanzados, las tareas asignadas al equipo de Colb.Ai con sus plazos correspondientes, y las tareas o acciones que corresponden al cliente. Este documento se genera y está disponible para ser enviado al cliente en el mismo momento en que finaliza la reunión, eliminando la latencia habitual en la producción de este tipo de documentación.

La skill aprovecha las capacidades nativas de Claude Cowork para acceder a herramientas de gestión de archivos en el entorno de escritorio, lo que le permite escribir directamente el documento en el directorio de trabajo del proyecto, integrándose de forma transparente en el flujo de gestión documental de la empresa.

12.2. Alternativa de código abierto

De forma análoga al enfoque adoptado en la toma de requerimientos, se dispone de una alternativa de código abierto e independiente para los casos en que no se desee depender de la plataforma Claude Cowork. Esta alternativa se basa en una aplicación que combina un modelo de lenguaje de gran tamaño, accedido mediante API, con la biblioteca python-docx para la generación de documentos Word con formato estructurado. El flujo de la aplicación es equivalente al de la skill: a partir de la transcripción de la reunión, el modelo genera los distintos apartados del documento de seguimiento con un formato definido, que son entonces volcados al documento Word mediante la biblioteca de generación. Aunque la integración es menos fluida que la ofrecida por Claude Cowork, esta alternativa es igualmente válida para los casos de uso habituales y no incurre en costes de suscripción adicionales más allá del consumo de la API del modelo.

13. Estado Actual de la Implementación

Este capítulo describe el estado de desarrollo de cada uno de los tres módulos del proyecto en el momento de redacción de esta memoria. El objetivo es ofrecer una fotografía honesta del grado de madurez de cada componente, distinguiendo entre lo que está operativo, lo que está en proceso iterativo y lo que queda pendiente como trabajo futuro.

13.1. Toma de Requerimientos

13.1.1. Módulo de transcripción

El módulo de transcripción para reuniones presenciales, basado en Azure Speech Service con diarización, está implementado e integrado en la aplicación. La transcripción automática para reuniones telemáticas no requiere implementación propia, dado que se delega en las herramientas nativas de las plataformas de videoconferencia (Microsoft Teams, Google Meet, Zoom).

13.1.2. Implementación propia (aplicación Python)

La aplicación Python de toma de requerimientos está en un estado avanzado de desarrollo. El pipeline completo está implementado y ha sido validado sobre un proyecto real de Colb.Ai con resultados satisfactorios: el sistema generó un plan de implementación estructurado con propuesta tecnológica justificada y lista de issues de GitHub accionables. Está pendiente una revisión final del código para consolidar casos límite y asegurar la robustez del módulo antes de considerarlo completamente terminado.

La skill de Claude Cowork para toma de requerimientos está igualmente operativa y ha sido utilizada en proyectos de la empresa.

Estado: Funcional. Pendiente de revisión y validación final.

13.2. Spec Driven Development: colbPowers

colbPowers está en un estado funcional. La infraestructura central del plugin está completa y operativa:

- El hook SessionStart inyecta correctamente el skill using-colbpowers al inicio de cada sesión, activando el comportamiento de orquestador en el agente principal.
- El conjunto de skills heredadas de Superpowers está disponible y funciona con la configuración de artefactos de colbSpec.
- El plugin ha sido instalado y probado en proyectos reales de Colb.Ai, refinándose de forma iterativa a partir del feedback de uso.

Las siguientes funcionalidades están pendientes de implementación:

Métricas de calidad de código: La integración de las métricas de Complejidad Ciclomática, Volumen de Halstead e Índice de Mantenibilidad en el flujo de revisión no está aún implementada. Requiere extender code-review-graph con los cálculos correspondientes sobre el AST de Tree-Sitter y exponer los resultados al agente Reviewer mediante una nueva herramienta MCP.

Adaptación completa de rutas de artefactos: Los skills brainstorming y writing-plans generan actualmente los documentos en la ruta heredada de Superpowers. La adaptación para que los artefactos aterricen directamente en .specs/memory/docs/specs/ y .specs/memory/docs/plans/ está pendiente como segunda iteración del plugin.

Estado: Funcional. En proceso iterativo activo. Métricas de calidad y adaptación de rutas de artefactos pendientes.

13.3. Seguimiento del Cliente

13.3.1. Skill de Claude Cowork

La skill de Claude Cowork para la generación automática de documentación de seguimiento post-reunión está implementada y en uso activo. Al finalizar cada reunión con el cliente, la skill procesa la transcripción y genera un documento Word estructurado con el resumen de los temas tratados, las decisiones acordadas y los próximos pasos para ambas partes. La skill ha sido utilizada en reuniones reales de Colb.Ai con resultado satisfactorio.

13.3.2. Alternativa propia

La implementación propia basada en python-docx (descrita en la sección 12.2) está pendiente de desarrollo. En el período cubierto por esta memoria, la skill de Claude Cowork ha cubierto completamente la necesidad operativa, por lo que la alternativa propia queda registrada como trabajo futuro.

Estado: Skill de Claude Cowork operativa. Alternativa propia pendiente de implementación.

Referencias

- [1] Ostroff, J., Makalsky, D., y Paige, R. (2004). Agile Specification-Driven Development. En: Extreme Programming and Agile Methods. Springer. https://doi.org/10.1007/978-3-540-24853-8_12. Accedido el 24/02/2026.
- [2] GitHub. (2025). github/spec-kit Repository. Recuperado de: <https://github.com/github/spec-kit>. Accedido el 24/02/2026.
- [3] Jama Software. (2026). What is Requirements Gathering in Software Engineering? Recuperado de: <https://www.jamasoftware.com/requirements-management-guide/>. Accedido el 25/02/2026.
- [4] Microsoft. (2026). Using agents in Visual Studio Code. Recuperado de: <https://code.visualstudio.com/docs/copilot/agents/overview>. Accedido el 25/02/2026.
- [5] AgentSkills. (2026). AgentSkills Documentation. Recuperado de: <https://agentskills.io/home>. Accedido el 25/02/2026.
- [6] Builder Methods. (2025). buildermethods/agent-os. Recuperado de: <https://github.com/buildermethods/agent-os>. Accedido el 25/02/2026.
- [7] GitHub. (2026). GitHub Copilot documentation. Recuperado de: <https://docs.github.com/en/copilot>. Accedido el 25/02/2026.
- [8] Cursor. (2026). Cursor: The best way to code with AI. Recuperado de: <https://cursor.com>. Accedido el 25/02/2026.
- [9] Departament d'Organització d'Empreses (FIB). Mòdul 2.1.b – Gestión ágil de proyectos. Universitat Politècnica de Catalunya. Accedido el 24/02/2026.
- [10] Indeed. (2026). Guía de salarios en España: Explorar sueldos por cargo y ubicación. Recuperado de: <https://es.indeed.com/career/salaries>. Accedido el 08/03/2026.
- [11] Barcelona Life. (2026). Guide to Coworking Spaces in Barcelona. Disponible en: <https://www.barcelona-life.com/barcelona-coworking-spaces>. Accedido el 10/03/2026.
- [12] Naturgy. (2026). Precio de la luz y tarifas eléctricas para el hogar. Disponible en: https://www.naturgy.es/hogar/luz/precio_de_la_luz. Accedido el 10/03/2026.
- [13] GitHub. (2026). GitHub Pricing Plans: Personal and Business. Recuperado de: <https://github.com/pricing>. Accedido el 09/03/2026.
- [14] Anthropic. (2026). Choosing a Claude plan: Free, Pro, and Team. Recuperado de: <https://support.claude.com/en/articles/11049762-choosing-a-claude-plan>. Accedido el 09/03/2026.
- [15] Anthropic. (2024). Claude Code: Agentic coding in your terminal. Recuperado de: <https://docs.anthropic.com/en/docs/claude-code>. Accedido el 29/04/2026.
- [16] OpenCode. (2025). OpenCode: Open source coding assistant. Recuperado de: <https://github.com/sst/opencode>. Accedido el 29/04/2026.

- [17] code-review-graph. (2025). Repositorio code-review-graph. Recuperado de: <https://github.com/nicholasgriffintn/llm-code-review-graph>. Accedido el 29/04/2026.
- [18] Superpowers. (2025). Superpowers plugin for Claude Code. Recuperado de: <https://github.com/superpowers-ai/superpowers>. Accedido el 29/04/2026.
- [19] McCabe, T. J. (1976). A Complexity Measure. IEEE Transactions on Software Engineering, 2(4), 308-320. <https://doi.org/10.1109/TSE.1976.233837>.
- [20] Oman, P., y Hagemester, J. (1992). Metrics for assessing a software system's maintainability. En: Proceedings Conference on Software Maintenance. <https://doi.org/10.1109/ICSM.1992.242525>.
- [21] Tavily. (2025). Tavily AI Search API. Recuperado de: <https://tavily.com>. Accedido el 29/04/2026.
- [22] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>. Accedido el 29/04/2026.